T21: wide producer arithmetic cuts at n = 73, K = 600 (scan gate) and the process-start factorise cut

Class: **engineering**, observation only. Same 4 vCPU Xeon 2.80 GHz KVM guest as T20 (49 isolated runs, 0 flagged contended; scan on cores 1 to 3). Protocol frozen before timing in `protocol.json` (commit 8519132, binary sha256 cb9502a2…). Raw records `runs/`, derived values `reports/analysis.json` (by `examples/t21_scan_analysis.rs`). Stacked on PR #2318.

Question and frozen rule

Do three cuts to the wide producer's per-probe arithmetic (card `NEXT_T21.md`) take the 20,000-state scan at the real K = 600 base to at least 1.2x? A cut (or all cuts together) clears only if the median ns-per-probe ratio to the control is at most 0.8333 and its 95% bootstrap upper bound at most 0.90; the end-to-end stage runs only if one clears. Same binary, switches set explicitly, 7 rounds, arm order rotated, A/A arm included.

Cuts (each with a unit test and a switch, default off)

- **sparse**: `Gf2_128` reduction by constant shifts for the tails {0,2,3,4} (n = 73) and {0,2,4,7} (n = 83), instead of two carry-less folds. Equal to the fold and the byte table on 20,000 random and edge inputs per modulus.
- **algebra**: one batched inversion of (l+r)lr; the quotients recovered by two squarings (10 multiplications plus 2 squarings per candidate instead of 11 multiplications). Same roots in the same order, scalar and batched, random and exceptional inputs, n = 73 and 83.
- **factorise**: early stop of the u128 trial division once the remainder is a word-size prime; same factor list as the old trial division (test includes the n = 73 order). Affects process start, not the scan.
- Dropped before the freeze: one `to_normal` per candidate through linearity (it needs `to_normal(q)`, so the table-apply count does not change).

Result: scan gate (stage A), ns per probe

arm	median ns/probe	ratio to ctl (95%)	speedup vs ctl	clears 1.2x
ctl (both off)	252.6	1	1	-
sparse	245.6	0.972 (0.948 to 1.141)	1.03	no
algebra	264.9	1.049 (0.940 to 1.193)	0.95	no
all	243.1	0.962 (0.933 to 1.143)	1.04	no
A/A ctl2	240.9	0.954 (0.921 to 1.082)	1.05	-

Probes and hits are identical in all 35 runs. The A/A arm moves by 4.6% and every cut sits inside that spread: **no cut reaches 1.2x on the scan, and none is distinguishable from the control at this resolution.** Per the frozen rule the end-to-end stage was **not run**; the negative scan result is the T21 result for these cuts. No S claim (counted S is identical by construction).

Deterministic instruction counts (stage C, secondary, K = 40, 2000 states, callgrind)

Collected Ir: ctl 2,631.6 M; sparse 2,605.7 M (-1.0%); algebra 2,718.7 M (+3.3%); all 2,690.4 M (+2.2%). `reduce_fold` alone: 424 M (ctl). The sparse reduction does not execute fewer instructions in the reduction itself; the algebra replaces one multiplication by two squarings (each a full carry-less product here) and executes more. A likely reason, not measured here (no per-instruction breakdown was taken): the carry-less reduction's instruction cost is mostly the moves between general and vector registers around its three products, which constant shifts on u128 words do not remove.

Process start (stage P): the factorise cut

Old binary (T20 head) against new, one core, `construct:73:1:40`, exit after one scanned state, 5 rounds alternating: old 8.06 to 8.69 s, new 0.19 to 0.25 s; new/old 0.024 (95% 0.023 to 0.031), 8.1 s saved per process. No A/A arm was run for this stage; the gap is about 40x, far outside any run-to-run spread seen here. The saving is a fixed cost per process start (about 0.2% of the T20 landed row's 4,600 s, a stage improvement for short runs and tests, not a pipeline improvement of the K = 600 row).

Limits and deviations

- Pilots during development (single and double scan runs, one callgrind pass) are excluded and disclosed in `protocol.json`; they set the defaults to off.
- The scan resolves differences above roughly 10%; a cut below that would not be seen. Instruction counts say there is none worth chasing for sparse and algebra.
- The factorise early stop only helps while the remainder after the cofactor fits a word (n = 73); the n = 83 order still needs a u128 primality test and a cofactor split (a card, not done here).
- Hardware: no VPCLMULQDQ on this host (AVX-512 F, BW, DQ, VL present), so a vectorised carry-less multiply across candidates was not testable here.

Next

T22 card: keep the u128 field's reduction in vector registers end to end (the u64 type's `fold_shifted` already does), since `reduce_fold` is 16% of the collected Ir of the K = 40 profile (hypothesis: mostly register moves, to be checked with a per-

instruction profile first); kill under 1.2x on the same scan gate. T23: u128 Miller-Rabin and a small-prime cofactor split so `KoblitzCurve::new` is fast at n = 83 and above.

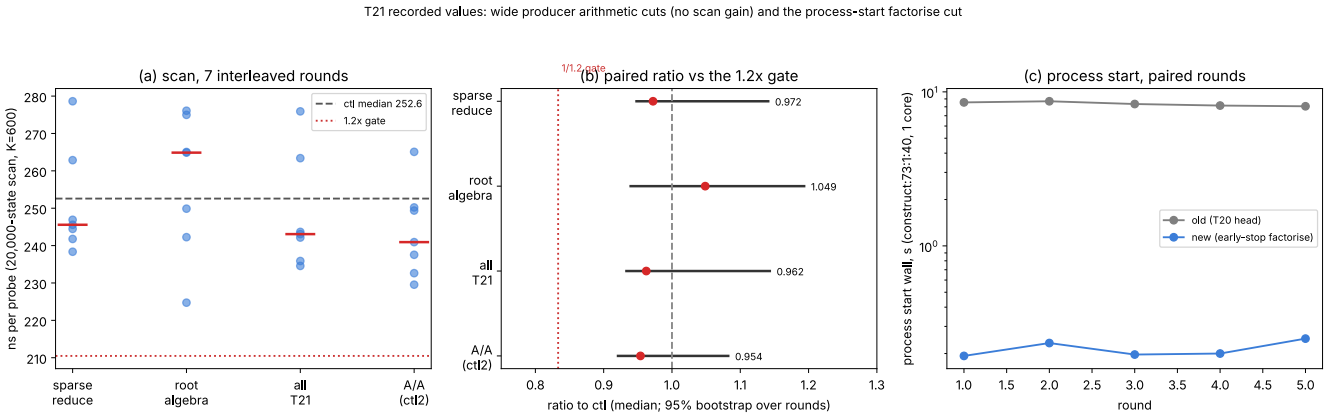


Figure 1: T21 recorded values: (a) scan ns per probe by round, (b) paired ratio against the 1.2x gate, (c) process-start wall old versus new. Source `reports/analysis.json`, `runs/`; plotted by `make_graph.py`.